

**EMBEDDED SYSTEM WITH IOT
EXPERIMENT NO. 5**

**IoT based Water Monitoring
System**

Name: NAVARRO, ROD GERYK C.

Course/Section: CPE162P-4/E01

Date of Performance: 05/26/2025

Date of Submission: 05/30/2025

CYREL O. MANLISES, PH.D.
Instructor

DISCUSSION

Connections

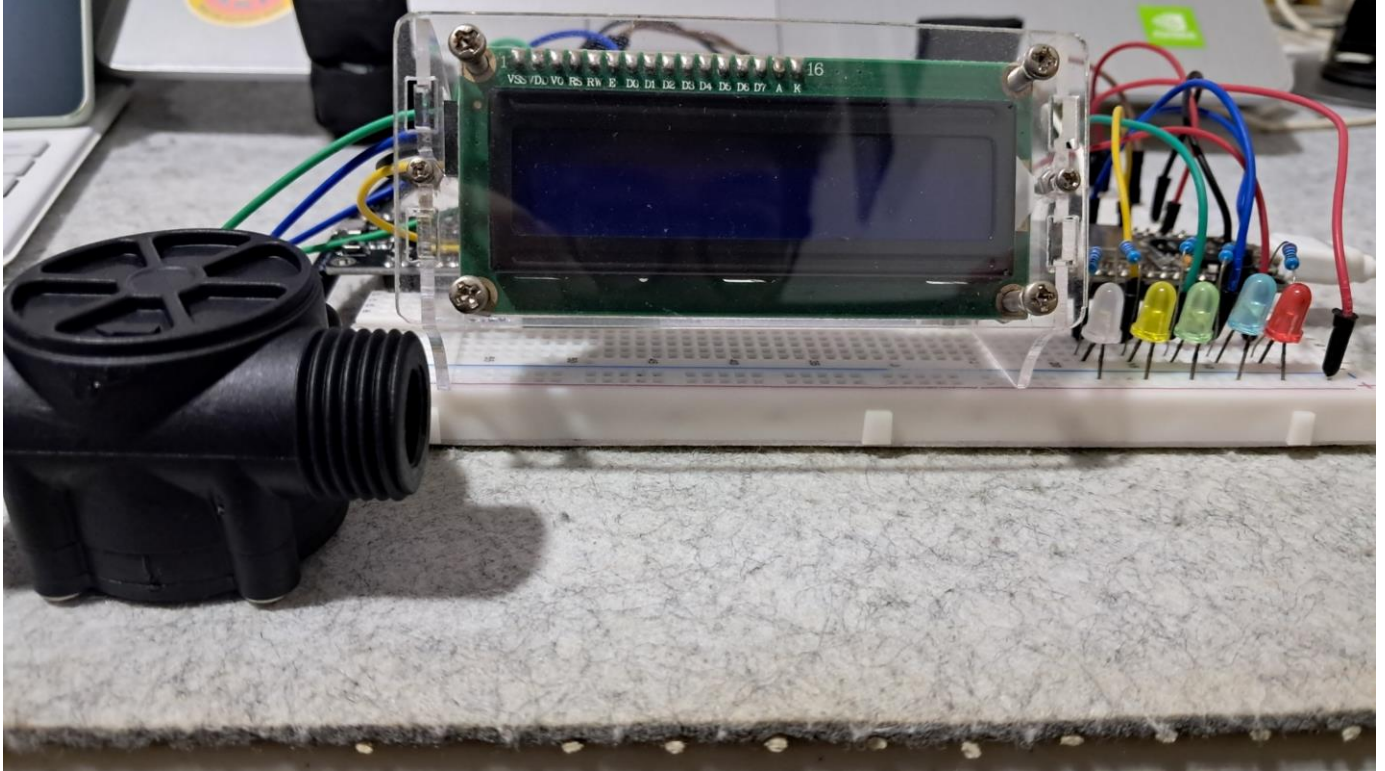


FIGURE 1: The Connection of the Water Flow Sensor, LEDs, and LCD to the ESP32 Dev Module

In this experiment, I created an IoT-based water monitoring system that measures the volume of water flowing through tubes by using a water flow sensor. The sensor counts pulses, which are converted into a flow rate measured in liters per minute (L/min). The formula used is:

Calculate flow rate

```
float newFlowRate = 2.663 * pulseCount / 1000 * 30; // Liters per minute
```

Where 2.663 is the calibration factor in milliliters per pulse. Dividing by 1000 converts the value to liters, and multiplying by 30 adjusts the 2-second reading to match one full minute (since there are 30 intervals of 2

seconds in a minute). The total volume of water is then calculated using the formula:

Calculate total volume

```
totalVolume += newFlowRate * (CALCULATION_INTERVAL / 60000.0);
```

Which adds the flow rate over time to get the overall volume in liters.

The materials I used in this experiment include an LCD to display the flow rate, total volume, and a timer. The timer starts when the sensor detects water flow and stops when the flow ends, showing how long water was running. I also used a water flow sensor to collect the data and five LEDs to show different volume levels of water, giving a quick visual reference. The system is controlled by the ESP32 Dev Module, which has built-in Wi-Fi. This allowed me to send the water flow and volume data to the internet using ThingSpeak, a platform for real-time monitoring. I connected all components to the ESP32 and programmed the system using the Arduino IDE, where I wrote the code to manage the sensor readings, display updates, LED indicators, and data transmission. Through this project, I was able to monitor water usage in real time, both on the device and online.

Code Development

Navarro_EXP5_CPE162P.ino

```
1  #include <LiquidCrystal_I2C.h>
2  #include <WiFi.h>
3  #include <ThingSpeak.h>
4  #include <time.h>
5
6  // WiFi Configuration
7  const char* ssid = "PLDTHOMEFIBRt6ucX";
8  const char* password = "PLDTWIFIItLK72";
9
10 // Thingspeak Configuration
11 const long myChannelNumber = 2964917;
12 const char myWriteAPIKey[] = "18A6ZBU4I3OLZZ44";
13
14 // Hardware Configuration
15 const int FLOW_SENSOR_PIN = 13;
16 const int WHITE_LED = 19;
17 const int GREEN_LED = 18;
18 const int BLUE_LED = 5;
19 const int YELLOW_LED = 4;
20 const int RED_LED = 23;
21
22 // Flow sensor parameters
23 volatile long pulseCount = 0;
24 float flowRate = 0.0;
25 float totalVolume = 0.0;
26 unsigned long lastTime = 0;
27 const int CALCULATION_INTERVAL = 2000;
28
29 // Volume thresholds
30 const float VOLUME_LOW = 5.0;
31 const float VOLUME_MODERATE = 10.0;
32 const float VOLUME_HIGH = 15.0;
33 const float VOLUME_MAX = 20.0;
34
35 // LCD Object
36 LiquidCrystal_I2C lcd(0x27, 16, 2);
37
38 // Time configuration
```

Navarro_EXP5_CPE162P.ino

```
38 // Time configuration
39 const char* ntpServer = "pool.ntp.org";
40 const long gmtoffset_sec = 8 * 3600; // GMT+8 for Philippines
41 const int daylightOffset_sec = 0;
42
43 // System Parameters
44 const int UPDATE_INTERVAL = 15000;
45 unsigned long lastUpdate = 0;
46 WiFiClient client;
47
48 // Flow tracking variables
49 char flowStartTime[9] = "00:00:00"; // Time when flow started
50 char flowEndTime[9] = "00:00:00"; // Time when flow ended
51 bool isFlowing = false; // Current flow state
52 bool wasFlowing = false; // Previous flow state
53 unsigned long lastTimeSync = 0;
54 const int TIME_SYNC_INTERVAL = 3600000; // Sync time every hour
55
56 // Timer variables
57 unsigned long flowStartMillis = 0; // When flow started (millis)
58 unsigned long flowDuration = 0; // Total flow duration in ms
59 char durationString[9] = "00:00:00"; // Formatted duration
60
61 // Average flow rate calculation
62 const int AVG_INTERVAL = 60000; // 1 minute in ms
63 float flowRateSum = 0.0; // Sum of flow rates
64 int flowRateCount = 0; // Number of samples
65 unsigned long lastAvgReset = 0; // Last time average was reset
66 float averageFlowRate = 0.0; // Calculated average
67
```

FIGURE 2: The Initialization of Variables and Components

In this part of the code, I declared all the variables and settings needed for the system to work. I included the Wi-Fi name and password so the ESP32 can connect to the internet. I also added the channel number and API key for ThingSpeak, which will receive and show the data online. Then, I set up the pins for the water flow sensor and the five LEDs. I used volatile long pulseCount to keep track of the number of pulses from the sensor, which tells me how much water is flowing. I also included variables to calculate the flow rate, total volume of water, and flow duration. Time settings for syncing the correct clock and variables for storing the start and end times of water flow were also added. This setup helps keep everything organized and makes the code easier to understand.

```

Navarro_EXP5_CPE162P.ino
68 void setup() {
69     initializePins();
70     initializeDisplay();
71     initializeSerial();
72     showStartupScreen();
73     connectToWiFi();
74     syncTime();
75     ThingSpeak.begin(client);
76
77     attachInterrupt(digitalPinToInterrupt(FLOW_SENSOR_PIN), pulseCounter, RISING);
78
79     // Initialize average timing
80     lastAvgReset = millis();
81 }
82

```

FIGURE 3: The Setup of the Components

In the setup() function, I started by calling different initialization functions for the pins, LCD display, and serial monitor. I also showed a startup screen on the LCD to let the user know the system is starting. After that, the ESP32 connects to Wi-Fi using the credentials provided, and it syncs the time from the internet using an NTP server. This time will be used to record when the water flow starts and ends. I also began the ThingSpeak connection so it can send data online. Lastly, I attached an interrupt to the flow sensor pin so every time a pulse is detected (meaning water flows), it increases the pulse count. This setup makes sure all components are ready before the loop starts running.

```

Navarro_EXP5_CPE162P.ino
83 void loop() {
84     // Sync time periodically
85     if (millis() - lastTimeSync > TIME_SYNC_INTERVAL) {
86         syncTime();
87     }
88
89     calculateFlowRate();
90     updateFlowTracking();
91     calculateAverageFlowRate(); // New function for average calculation
92     updateDisplay();
93     updateLEDs();
94     sendToCloud();
95     delay(100);
96 }
97

```

FIGURE 4: The Loop Function

The `loop()` function is the heart of the program that keeps running continuously. It starts by checking if the system needs to sync the time again, which happens once every hour. Then it calculates the current flow rate and updates the water volume. The program also tracks whether water is currently flowing and records the start and stop times. It calculates the average flow rate over every 1 minute and shows all the information (flow rate, total volume, and flow time) on the LCD. The LEDs are also updated based on how much water has flowed so far, giving a visual indicator of the volume level. Finally, the system sends the latest data to ThingSpeak so I can view it online. The loop runs with a short delay of 100 milliseconds to keep the updates smooth.

```

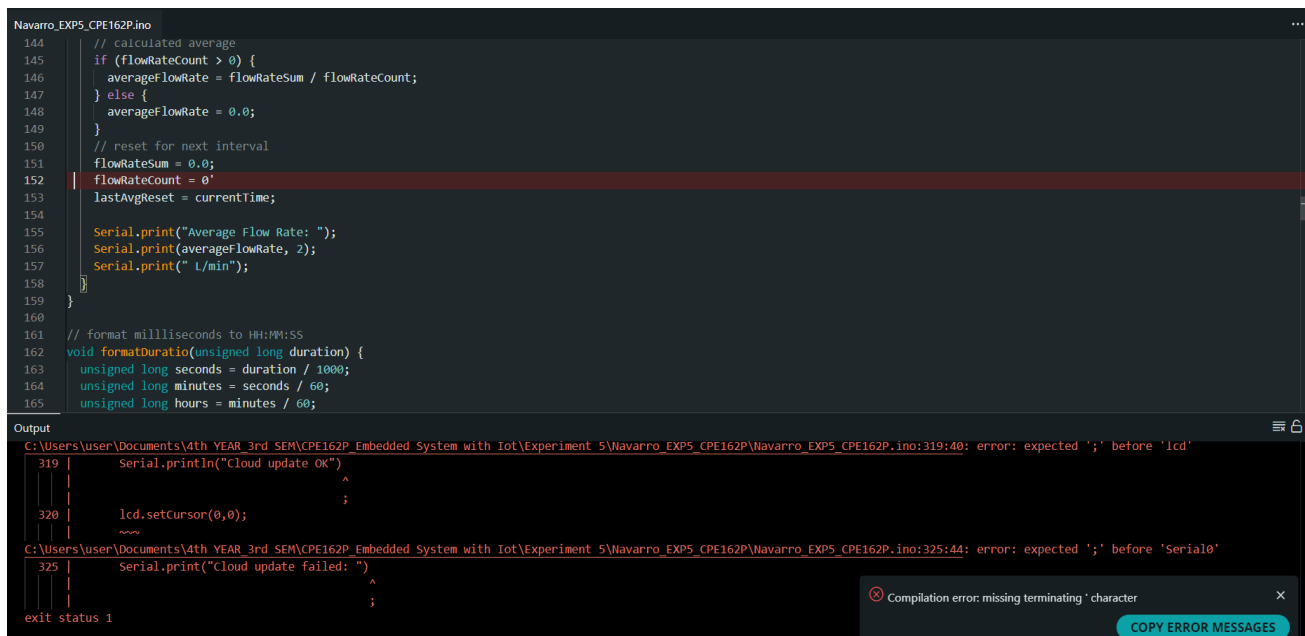
Navarro_EXP5_CPE162P.ino
96 }
97
98 // Time synchronization function
99 void syncTime() {
100     configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
101     lastTimeSync = millis();
102     Serial.println("Time synchronized");
103 }
104
Navarro_EXP5_CPE162P.ino
286 }
287
288 // LED Control
289 void updateLEDs() {
290     allLEDsOff();
291
292     if (totalVolume < VOLUME_LOW) {
293         digitalWrite(WHITE_LED, HIGH);
294     } else if (totalVolume < VOLUME_MODERATE) {
295         digitalWrite(YELLOW_LED, HIGH);
296     } else if (totalVolume < VOLUME_HIGH) {
297         digitalWrite(GREEN_LED, HIGH);
298     } else if (totalVolume < VOLUME_MAX) {
299         digitalWrite(BLUE_LED, HIGH);
300     } else {
301         digitalWrite(RED_LED, HIGH);
302     }
303 }
304
Navarro_EXP5_CPE162P.ino
175 void updateDisplay() {
176     static bool firstRun = true;
177     static char lastStatusline[17] = "";
178     static char lastDataLine[17] = "";
179
180     // Build status line
181     char statusline[17];
182     if (isFlowing) {
183         // Update duration during flow
184         unsigned long currentDuration = millis() - flowStartTime;
185         formatDuration(currentDuration);
186         sprintf(statusline, sizeof(statusline), "Flowing %s", durationString);
187     } else {
188         sprintf(statusline, sizeof(statusline), "Last D: %s", durationString); // 10 =
189     }
190
191     // Build data line
192     char dataLine[17];
193     sprintf(dataLine, sizeof(dataLine), "F: %s, V: %s, T: %s", flowRate, totalVolume);
194
195     // Update only changed portions of the display
196     if (firstRun || strcmp(statusline, lastStatusline) != 0) {
197         lcd.setCursor(0, 0);
198         lcd.print(statusline);
199         strcpy(lastStatusline, statusline);
200     }
201
202     if (firstRun || strcmp(dataLine, lastDataLine) != 0) {
203         lcd.setCursor(0, 1);
204         lcd.print(dataLine);
205         strcpy(lastDataLine, dataLine);
206     }
207
208     firstRun = false;
209
210     // debugging output (every 5 seconds)
211     static unsigned long lastSerialOutput = 0;
212     if (millis() - lastSerialOutput > 5000) {
213
Navarro_EXP5_CPE162P.ino
267 void calculateFlowRate() {
268     static unsigned long lastCalculation = 0;
269     unsigned long currentTime = millis();
270
271     if ((currentTime - lastCalculation) > CALCULATION_INTERVAL) {
272         lastCalculation = currentTime;
273
274         // Calculate flow rate
275         float newFlowRate = 2.663 * pulseCount / 1000 * 30; // Liters per minute
276
277         // Calculate total volume
278         totalVolume += newFlowRate * (CALCULATION_INTERVAL / 60000.0);
279
280         // Apply smoothing to flow rate
281         flowRate = (flowRate * 0.7) + (newFlowRate * 0.3);
282
283         // Reset pulse counter
284         pulseCount = 0;
285     }
286 }
287
Navarro_EXP5_CPE162P.ino
161 // Format milliseconds to HH:MM:SS
162 void formatDuration(unsigned long duration) {
163     unsigned long seconds = duration / 1000;
164     unsigned long minutes = seconds / 60;
165     unsigned long hours = minutes / 60;
166
167     seconds %= 60;
168     minutes %= 60;
169     hours %= 100; // Limit to 2 digits
170
171     sprintf(durationString, sizeof(durationString), "%02lu:%02lu:%02lu", hours, minutes, seconds);
172 }
Navarro_EXP5_CPE162P.ino
263 void pulseCounter() {
264     pulseCount++;
265 }

```

FIGURE 5: Other Important Functions

There are several helper functions that make the program easier to manage. The `syncTime()` function gets the correct time from the internet, which is useful for recording the exact time when water starts or stops flowing. The `pulseCounter()` function increases the pulse count whenever the flow sensor detects water movement. The `calculateFlowRate()` function converts the number of pulses into liters per minute using the formula mentioned earlier. It also updates the total water volume. The `updateFlowTracking()` function checks if water is currently flowing and logs the start or end time. The `formatDuration()` function changes the duration from milliseconds into hours, minutes, and seconds so it can be displayed nicely. The `updateDisplay()` function shows all the data on the LCD screen, while `updateLEDs()` turns on the right LED depending on how much water has passed through. Each of these functions helps keep the main code clean and organized while handling different tasks in the system.

Errors Encountered during Code Development



The screenshot displays an IDE window titled "Navarro_EXP5_CPE162P.ino". The code editor shows lines 144 through 165. Lines 144-159 define a function to calculate the average flow rate, and lines 161-165 define a function to format duration in HH:MM:SS. The "Output" window at the bottom shows two compilation errors: "error: expected ';' before 'lcd'" at line 319 and "error: expected ';' before 'Serial0'" at line 325. A red squiggly line is visible under the semicolon at line 325. A "Compilation error: missing terminating ' character" message box is also present in the bottom right corner.

```
Navarro_EXP5_CPE162P.ino
144 // calculated average
145 if (flowRateCount > 0) {
146     averageFlowRate = flowRateSum / flowRateCount;
147 } else {
148     averageFlowRate = 0.0;
149 }
150 // reset for next interval
151 flowRateSum = 0.0;
152 flowRateCount = 0;
153 lastAvgReset = currentTime;
154
155 Serial.print("Average Flow Rate: ");
156 Serial.print(averageFlowRate, 2);
157 Serial.print(" L/min");
158 }
159 }
160
161 // format milliseconds to HH:MM:SS
162 void formatDuration(unsigned long duration) {
163     unsigned long seconds = duration / 1000;
164     unsigned long minutes = seconds / 60;
165     unsigned long hours = minutes / 60;
166 }

Output
C:\Users\user\Documents\4th YEAR 3rd SEM\CPE162P Embedded System with Iot\Experiment 5\Navarro_EXP5_CPE162P\Navarro_EXP5_CPE162P.ino:319:40: error: expected ';' before 'lcd'
319 |         Serial.println("Cloud update OK");
    |                                ^
320 |         lcd.setCursor(0,0);
    |         ~~~
C:\Users\user\Documents\4th YEAR 3rd SEM\CPE162P Embedded System with Iot\Experiment 5\Navarro_EXP5_CPE162P\Navarro_EXP5_CPE162P.ino:325:44: error: expected ';' before 'Serial0'
325 |         Serial.print("Cloud update failed: ")
    |                                            ^
exit status 1
Compilation error: missing terminating ' character
COPY ERROR MESSAGES
```

FIGURE 6: Error 1

The error shown on Figure 6 the message "Cloud update OK" is not properly closed, which confuses the compiler and results in multiple syntax errors. This also leads to a cascade of further errors in the subsequent lines, such as `expected ; before 'lcd` and `expected ; before 'Serial` on lines 320 and 325, because the compiler cannot correctly interpret the following code once a string is left open. Fixing this requires adding semicolon (;) at the end of the affected `Serial.println()` line to properly terminate the statement.

```

Navarrio_EXP5_CPE162P.ino
82 }
83
84 void loop() {
85   //sync time periodically
86   if (millis() - lastTimeSync > TIME_SYNC_INTERVAL) {
87     syncTime();
88   }
89
90   calculateFlowRate();
91   updateFlowTracking();
92   calculateAveragaFlowRate();
93   updateDisplay();
94   updateEDs();
95   sendToCloud();
96   delay(100);
97 }
98
99 //time synchronization function
100 void syncTime() {
101   configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
102   lastTimeSync = millis();
103 }

```

Output

```

C:\Users\User\Documents\4th YEAR_3rd SEM\CPE162P Embedded System with Iot\Experiment 5\Navarrio_EXP5_CPE162P\Navarrio_EXP5_CPE162P.ino:199:28: error: 'stausLine' was not declared in this scope
199 | strcpy(lastStatusline, stausLine);
    |                            ^~~~~~
    |                            statusline
C:\Users\User\Documents\4th YEAR_3rd SEM\CPE162P Embedded System with Iot\Experiment 5\Navarrio_EXP5_CPE162P\Navarrio_EXP5_CPE162P.ino: In function 'void sendToCloud()':
C:\Users\User\Documents\4th YEAR_3rd SEM\CPE162P Embedded System with Iot\Experiment 5\Navarrio_EXP5_CPE162P\Navarrio_EXP5_CPE162P.ino:326:22: error: 'Status' was not declared in this scope; did you mean 'lastTimeSync'?
326 | Serial.println(Status);
    |                  ^~~~~
    |                  status
exit status 1

Compilation error: 'calculateAveragaFlowRate' was not declared in this scope; did you mean 'calculateAverageFlowRate'?

```

FIGURE 7: Error 2

The error shown in the image is caused by a typographical mistake in the function name `calculateAveragaFlowRate` on line 92 of the `loop()` function. The correct function name is `calculateAverageFlowRate`, and the misspelling causes the compiler to throw the error `was not declared in this scope`, meaning it cannot find a matching definition for the function. Additionally, other errors in the output point to misspelled variable names such as `stausLine` instead of `statusLine` on line 199, and `Status` instead of the correctly declared variable name (likely `status`) on line 326 inside the `sendToCloud()` function. These kinds of spelling errors prevent the code from compiling because the compiler treats names as case-sensitive and exact. Fixing these issues involves correcting the spelling of the function and variable names to match their declarations.

Difficult Aspect of the Experiment's Development

On the software side, the hardest part was creating the timer that shows on the LCD. I had to make many changes to the timer to get the result I wanted. The tricky part was setting up the timer to turn ON and OFF at the right time to show how long the water flows through the tubes. On the hardware side, the most challenging part was setting up the whole system. It made me think creatively about how to make everything work well using only the materials I had.

5 Different Tubes Used



FIGURE 8: Five Tubes

For the experiment, I used five different tube sizes to see if they would affect the water flow rate. Since I only had one size of straw, I used ball pen casings as substitutes for the other four tubes. I made sure that each of the four ball pen casings had a different size.

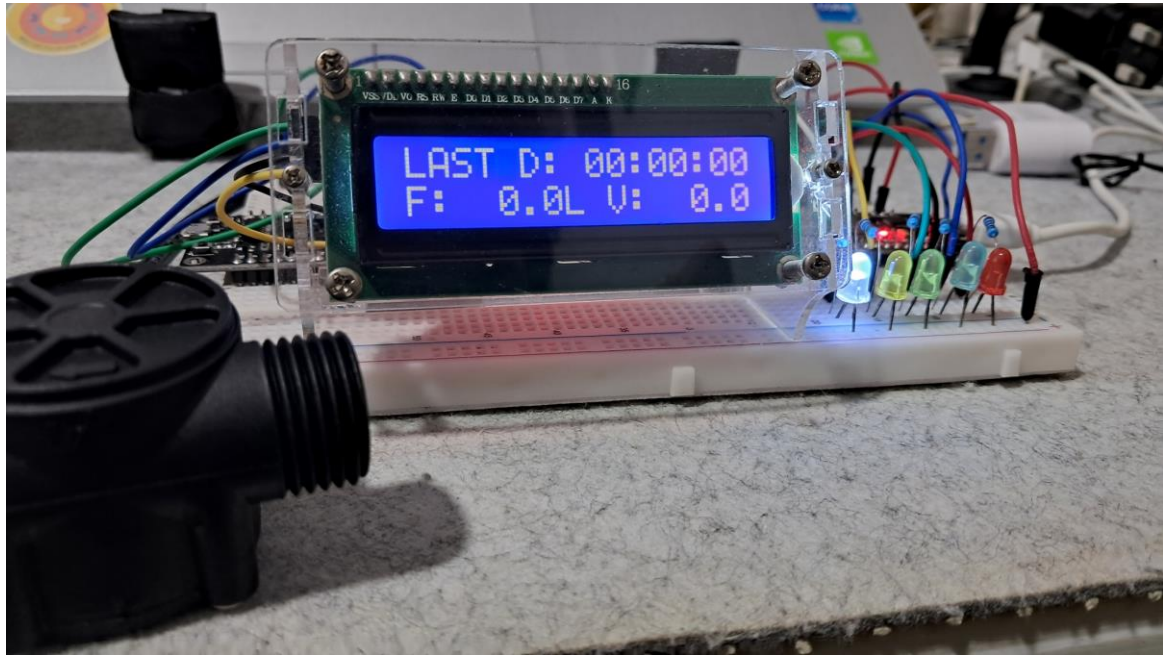


FIGURE 9: Working Prototype

This is the final working prototype of the IoT based water monitoring system experiment, which successfully meets all the requirements.

Interpretations of ThingSpeak's Graphs and Widgets

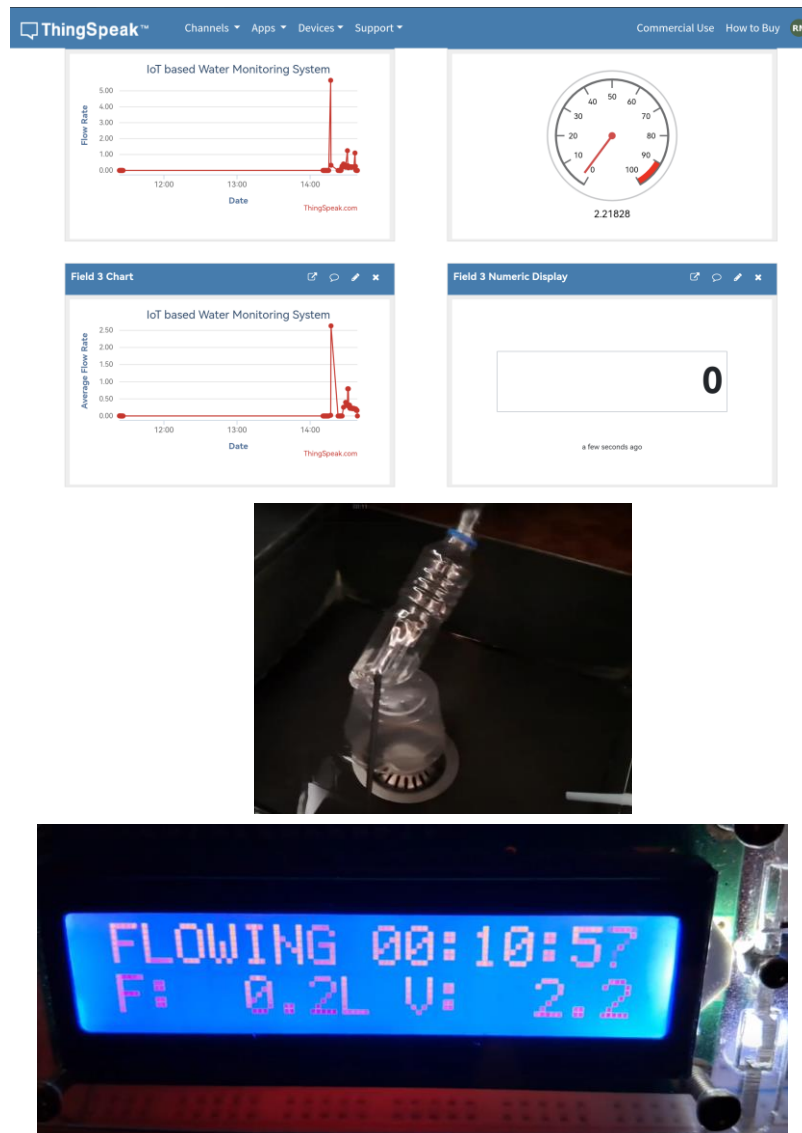


FIGURE 10: ThingSpeak Visualization: Data Gathered using the First Tube (smallest)

In the image above, water flows from the faucet into a plastic bottle, which fills up enough to send water through the smallest tube connected to the inlet of the water flow sensor. As expected, this tube results in a low flow rate, with most points on the graph ranging between 0 to 1 liter per minute, and one spike above 5 liters per minute due to an improper connection. The average flow rate stays between 0 to 0.5 liters per minute, showing consistently low flow, and the total volume after eleven minutes was only 2.2 liters. Since the volume is less than 5 liters, the white LED lights up to indicate volume less than 5 liters.



FIGURE 11: ThingSpeak Visualization: Data Gathered using the Second Tube

Looking at the data from the second tube, which is a ball pen casing slightly larger than the first one, we can see an increase in the water flow rate. The graph shows a small rise in flow, resulting in an average flow rate of about 1 liter per minute higher than the first tube. The yellow LED also lights up, showing that the water volume reached 5 liters.

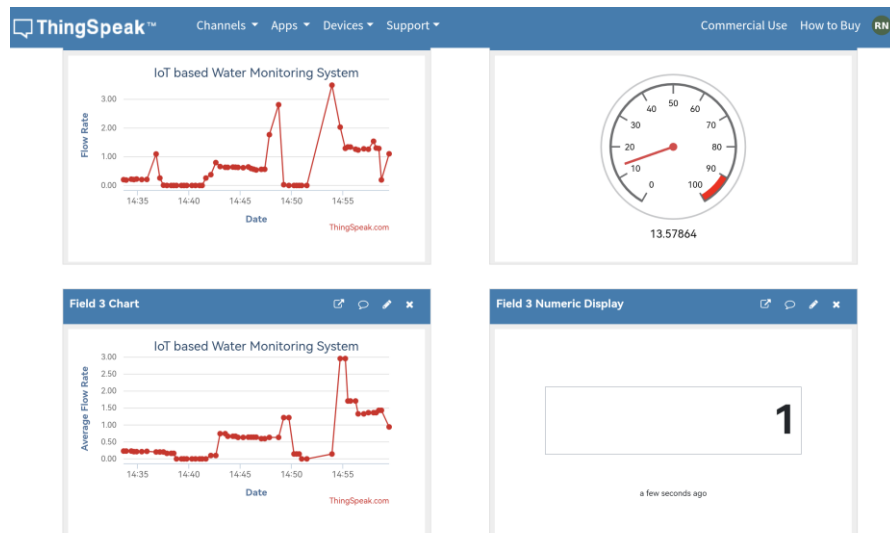


FIGURE 12: ThingSpeak Visualization: Data Gathered using the Third Tube

In this tube, more points in the flow rate graph become stable above one liter per minute compared to the previous tube. Note that the tube is slightly larger than the second tube. The average flow rate remains at one liter per minute but compared to the previous tube it has more stable points beyond 0.5 liter per minute. This means that this third tube has a better flow rate compared than the previous tubes. We can also see that the green LED lights up, showing that the water volume reached 10 liters.



FIGURE 13: ThingSpeak Visualization: Data Gathered using the Fourth Tube

The volume using the fourth tube went beyond the final volume level of 20 liters. The red LED turned on, indicating that the water reached this amount. With this tube, there was a big increase in the average flow rate, rising from 1 liter per minute to 3 liters per minute. The flow rate graph shows more stable and higher points above 1 liter per minute. This tube had a better flow rate than the previous ones, gaining 10 liters in just 5 minutes.

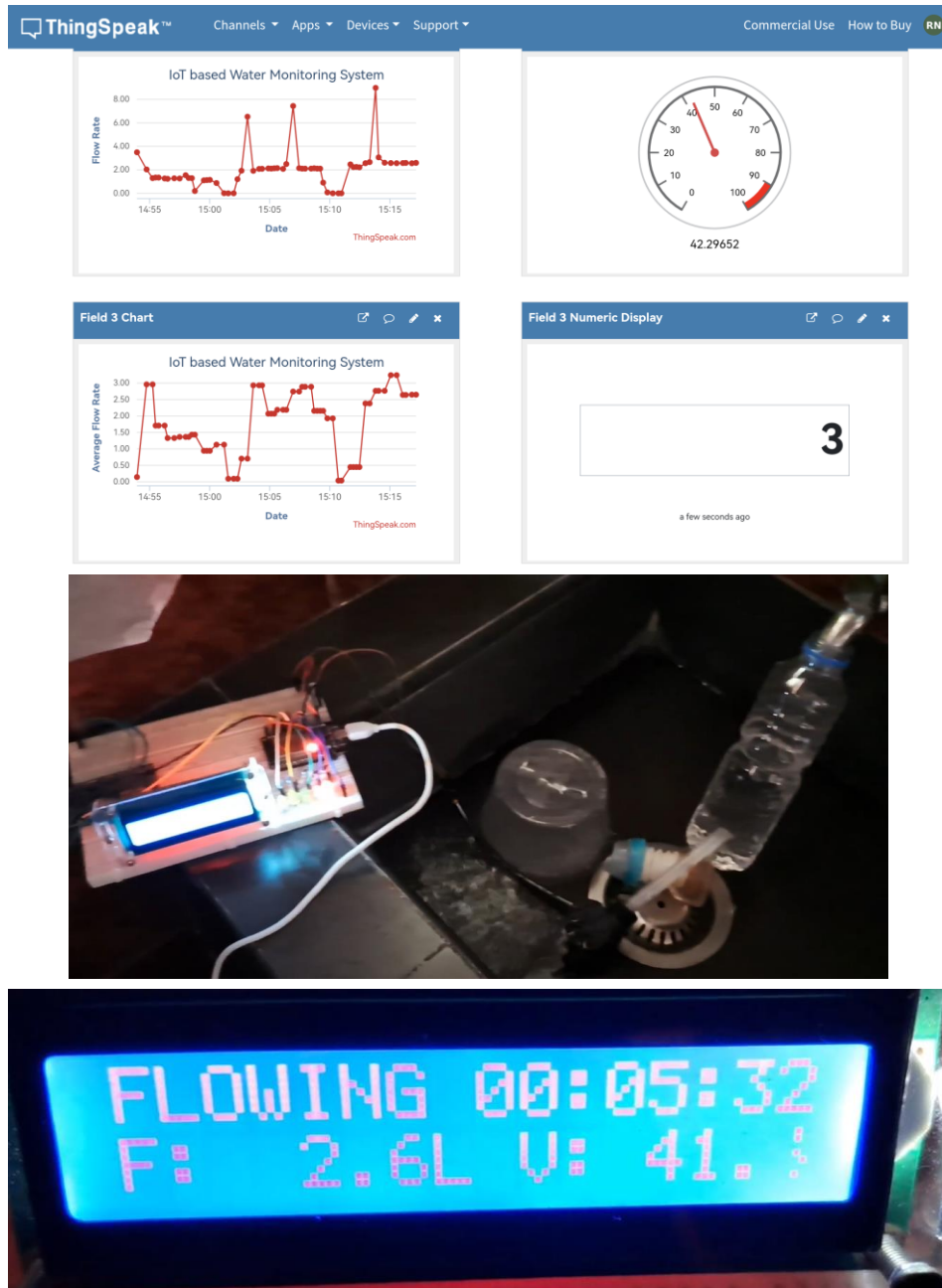


FIGURE 14: ThingSpeak Visualization: Data Gathered using the Fifth Tube (largest)

In the flow rate graph, we can see more stable points slightly above 2 liters per minute, which are higher than the values from the previous tubes. Although the average flow rate is still 3 liters per minute, just like the previous tube, this tube shows a more consistent flow. The graph shows that as the size of the tube increases, the flow rate also increases. However, it's also clear that water pressure plays an important role in increasing the flow rate through the tubes.

CONCLUSION

Working on this IoT-based water monitoring system, I learned a lot about both hardware and software. My goal was to build a system that could measure how much water flows through different tubes using a water flow sensor, and show the data on an LCD and through LED indicators. The ESP32 Dev Module was the brain of the system, helping me read data from the sensor, display it on the screen, and even send it online using ThingSpeak. I used five LED lights to show different volume levels so users could easily understand how much water has flowed just by looking. Connecting everything together—including the LCD, sensor, LEDs, and Wi-Fi setup—took careful planning, especially when testing with five different tube sizes. When I worked on the code, I faced a few challenges. One of the hardest parts was creating a timer that could correctly show how long the water was flowing. I had to make several changes before getting it right. I also encountered common errors like missing semicolons and misspelled function names, which caused the code not to run. By fixing these issues, I understood how small mistakes could affect the entire program. I organized the code by using helper functions like calculating flow rate, syncing time, and updating the LED indicators, which made everything easier to manage and debug. I also realized how useful the ESP32 is because it not only reads sensor data but also sends it to the internet so I can track water usage even when I'm not near the device.

Through this experiment, I discovered how tube size affects water flow rate. By using different sizes—including ballpen casings as substitutes—I observed changes in how fast water passed through the sensor. The smallest tube showed the lowest flow rate, while the larger ones gave more stable and higher readings. ThingSpeak graphs helped me clearly see the flow trends and confirm that my system was working properly. The final prototype showed accurate flow rates, volumes, and durations on the LCD and LEDs. Overall, this project gave me a deeper

understanding of IoT systems, coding logic, and real-world data monitoring, and I now feel more confident in developing similar systems in the future.